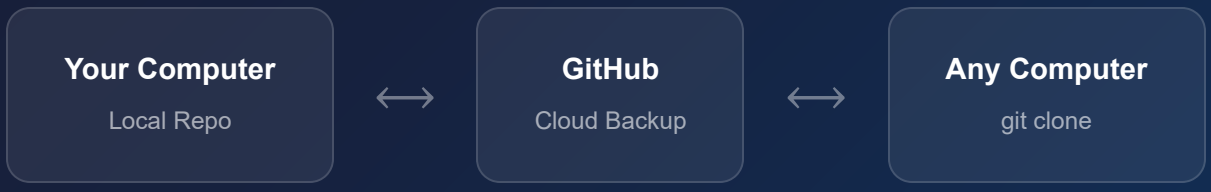
LESSON 01

Git & GitHub for New Developers

Version control, branching, cloning, and the mental models you need to work like a professional developer.



May 2026 6 Topics Hands-on Project

CHAPTER 1

Git vs GitHub

They sound the same but they do completely different things. Understanding this distinction is the most important first step.

📄 Git

A tool installed on **your computer** that tracks every change you make to your files. It runs entirely offline. Think of it as a very powerful undo history for your entire project.

☁️ GitHub

A **website** that stores a copy of your Git repository in the cloud. It lets you access your code from anywhere, share it with others, and back it up safely.

THE SIMPLE ANALOGY

Git is your save system. **GitHub** is the cloud backup of those saves.

How They Work Together

```
Your Computer GitHub (Cloud) ──────────────────────────────
────────────────────────── Files you edit Your repo online ↓ ↑ git add →
stage changes │ git commit → save a snapshot │ git push
────────────────────────────────────→ uploads snapshots git pull
←──────────────────────────────────── downloads snapshots
```

KEY MENTAL MODEL

Your local folder and your GitHub repo are **two separate copies**. Neither updates automatically — you control sync with `push` and `pull`.

CHAPTER 2

Version Control

Every time you commit, Git takes a snapshot of your project. You can always travel back to any snapshot.

What is a Commit?

A commit is a saved snapshot of all your files at a specific moment. It's like a save point in a video game — you can always go back to it. Each commit has a unique ID and a message describing what changed.

```
── Timeline ──────────────────────────────────────────────────────── ●
a1f4e6a "Initial commit: add Tic Tac Toe game" ↓ ● 2d3fa2b "Add CLAUDE.md
with project guidance" ↓ ● aa676ab "Add reset scores button" ↓ ● HEAD ← you
are here (latest commit)
```

The 3-Step Commit Workflow

1. **Make your changes**

   Edit files normally in your project folder.

2. **Stage the changes — `git add`**

   Tell Git which changed files you want to include in the next snapshot.

3. **Commit — `git commit -m "message"`**

   Take the snapshot. Write a clear message about what and why you changed.

```
# The everyday workflow
git add tictactoe.html          # stage one file
git add .                       # stage all changed files
```

```
git commit -m "Add dark mode"   # save the snapshot
git push                         # upload to GitHub
```

> **WHY THIS MATTERS**
>
> Git only tracks **what changed** between snapshots — not full file copies each time. This makes it extremely efficient even across thousands of commits.

CHAPTER 3

Push, Pull & File Recovery

Syncing between your computer and GitHub, and what actually happens to your files when you do.

git push

Sends your new local commits **up to GitHub**. Run this after committing to back up and share your work.

git pull

Downloads commits from GitHub **to your local folder**. Run this when you switch computers or a teammate pushed changes.

What Happens to Files When You Pull?

Git does **not** wipe your folder and reinstall everything. It surgically applies only what changed:

What changed on GitHub	What happens locally
A file was modified	Your copy gets the updated version
A new file was added	That file appears in your folder
A file was deleted	It gets removed from your folder
A file was unchanged	Git doesn't touch it

Recovering a Deleted File

We demonstrated this live: `tictactoe.html` was deleted from the local folder without committing the deletion.

```
Scenario: file deleted locally, NOT committed Local folder: tictactoe.html
MISSING Git snapshot: tictactoe.html SAFE ✓ git restore tictactoe.html ↓
Local folder: tictactoe.html RESTORED ✓
```

Scenario	Command
Deleted/changed file, not yet committed	`git restore <filename>`
Need commits from GitHub you don't have	`git pull`
Committed a mistake and pushed it	`git revert <commit-id>`

THE SAFETY RULE

As long as you haven't **committed** a bad change, you can always undo it instantly with `git restore`. The danger zone is only after committing — and even then, it's recoverable.

CHAPTER 4

Branching

Branches are parallel timelines of your project. They let you experiment freely without ever touching your stable, working version.

The Mental Model

Why Branching Matters

1. **Production stays safe**
   Master/main is your live, working version. Users always get the stable build.

2. **Fearless experimentation**
   Try anything on a branch. If it fails, delete the branch — master never knew it existed.

3. **Parallel development**
   Work on 3 features at once across 3 branches without them interfering with each other.

The Standard Two-Branch Setup

Branch	Purpose	Rule

`master` / `main`	Production — what users see	Never broken, always stable
`dev`	Development — where work happens	Can be messy, gets tested here

Branch Commands We Used

```
# Create a new branch and switch to it
git checkout -b dev

# Switch between branches
git checkout master
git checkout dev

# Push a new branch to GitHub
git push -u origin dev

# Merge dev into master (deploy)
git checkout master
git merge dev
git push
```

PULL REQUESTS — THE PROFESSIONAL MERGE

Instead of merging locally, open a **Pull Request on GitHub**. This lets you (and teammates) review changes visually before they hit production. Same outcome as `git merge`, but with a review step and a clean paper trail.

CHAPTER 5

Cloning & Forking

How to take someone else's GitHub repository and run it on your own computer.

git clone — Getting Any Repo Locally

`git clone` downloads a repository and creates a new folder for it automatically. You can clone to any location on your computer — the folder path almost never matters.

```
# Clone any public repository
git clone https://github.com/someuser/some-repo

# A folder called "some-repo" is created automatically
# No need to pre-create the folder yourself
```

Does the Folder Path Matter?

Situation	Path matters?
A web app / HTML file (like your Tic Tac Toe)	**No**
Node.js / Python / most apps	**No — uses relative paths internally**
Config file with a hardcoded path	**Sometimes — README will warn you**
System-level tools	**Rarely — installer handles it**

> **WHY PATHS RARELY MATTER**

> Code almost always uses **relative paths** like `./images/logo.png` — relative to where the file lives, not to a fixed location on your hard drive. Moving the whole folder anywhere still works.

Clone vs Fork

Clone

Copies the repo to your computer. You **cannot push changes back** to the original owner's repo (unless they give you permission).

Fork

Copies the repo to **your GitHub account** first. Then you clone your fork. Now you own it, can push freely, and can propose changes via a Pull Request.

```
── When to use each ──────────────────────────────────────── Just want to run it locally? → git clone <url> Want to make it your own / contribute back? → Fork on GitHub first → then git clone <your-fork-url>
```

The Clone Workflow

1. **Find the repo on GitHub**
   Copy the URL from the browser or the green "Code" button.

2. **Clone it**
   `git clone <url>` — run this from wherever you want the folder to appear.

3. **Follow the README**
   Install dependencies (`npm install` , `pip install` , etc.) and run the project as instructed.

CHAPTER 6

Summary & Cheat Sheet

Everything covered in this lesson in one place.

KEY MENTAL MODELS

- **Git** = save system on your computer. **GitHub** = cloud backup of those saves.
- A **commit** is a snapshot. You build a timeline of snapshots. You can always go back.
- Your local folder and GitHub are **two separate copies**. You sync them with push/pull.
- Pull does **not** wipe your folder — it surgically applies only what changed.
- Branches are **parallel timelines**. Master = production (stable). Dev = where you work.
- Folder **path almost never matters** — code uses relative paths internally.
- **Clone** = run it locally. **Fork** = own a copy on GitHub, then clone your fork.

Command Cheat Sheet

Command	What it does
`git init`	Start a new git repo in the current folder
`git status`	See what files have changed
`git add <file>`	Stage a file for the next commit
`git commit -m "msg"`	Save a snapshot with a message

`git push`	Upload commits to GitHub
`git pull`	Download commits from GitHub
`git restore <file>`	Undo uncommitted changes to a file
`git checkout -b <name>`	Create and switch to a new branch
`git merge <branch>`	Merge a branch into the current one
`git clone <url>`	Download any repository locally
`git log --oneline`	View commit history (compact)

THIS SESSION'S PROJECT

We built a Tic Tac Toe game, initialized a Git repo, pushed it to GitHub, practiced branch-based development by adding a "Reset Scores" feature on a `dev` branch, and demonstrated live file recovery with `git restore`. Repo: **github.com/AdamVisutag/ClaudeTut**